

KCPC Week 3 Solutions

The Lambda Calculus: Functions, Computation, and Numbers

These solutions accompany all 12 problems. Equalities $\equiv_\beta$ mean beta-convertibility (up to alpha-renaming), not necessarily one reduction step. Every named function is a shorthand for a *closed pure lambda term*. Python and C++ snippets demonstrate the same behaviour; the proofs establish correctness for arbitrary terms, beyond those executable examples.

1 Foundational warm-ups

1. Terms and parentheses

Key insight: An abstraction binds its entire body; application groups to the left. The full grammar writes the term as

$$(\lambda x. (\lambda y. (x(yz)))).$$

$FV(yz) = \{y, z\}$, $FV(x(yz)) = \{x, y, z\}$, and removing y and then x under the two abstractions gives $FV = \{z\}$. The root of $\lambda x.xy$ is an abstraction; its body (xy) is an application.

2. Free variables and substitution

Key insight: Binding removes a variable from the free-variable set, not from the term. By structural definition,

$$FV(x) = \{x\}, \quad FV(MN) = FV(M) \cup FV(N), \quad FV(\lambda x.M) = FV(M) \setminus \{x\}.$$

Thus $FV((\lambda x.M)N) = (FV(M) \setminus \{x\}) \cup FV(N)$. For $(\lambda x.xy)z$ this is $(\{x, y\} \setminus \{x\}) \cup \{z\} = \{y, z\}$. The x in xy lies inside λx and is bound by it; substitution for a different free x cannot reach that occurrence.

3. Alpha conversion and capture

Key insight: A legal rename preserves which binder owns each occurrence. Rename the outer x to fresh u , and the inner y to fresh v , obtaining $\lambda u.\lambda v.uv$. By contrast, $\lambda y.\lambda y.yy$ binds both occurrences of y at the inner binder; the first term has one occurrence belonging to each binder. Alpha-renaming cannot change that ownership. For beta reduction, first rename the inner binder:

$$(\lambda x.\lambda y.x)y \equiv_\alpha (\lambda x.\lambda z.x)y \rightarrow_\beta \lambda z.y.$$

y remains free. The incorrect $\lambda y.y$ binds it and has no free variables, so cannot be the same result.

4. Identity and constant functions

Key insight: An unused bound argument vanishes when its binder is applied.

$$IA = (\lambda x.x)A \rightarrow_\beta A, \quad KAB = (\lambda x.\lambda y.x)AB \rightarrow_\beta (\lambda y.A)B \rightarrow_\beta A,$$

$$FAB = (\lambda x. \lambda y. y)AB \rightarrow_{\beta} (\lambda y. y)B \rightarrow_{\beta} B.$$

Both selectors require two beta steps. In particular $KII \rightarrow_{\beta} (\lambda y. I)I \rightarrow_{\beta} I$. If the substituted argument has free variables, alpha-rename an inner binder first as in Problem 3.

5. Currying and composition

Key insight: Applying one argument returns a function awaiting the next.

$$Cfga = (\lambda f. \lambda g. \lambda x. f(gx))fga \rightarrow_{\beta} (\lambda g. \lambda x. f(gx))ga \rightarrow_{\beta} (\lambda x. f(gx))a \rightarrow_{\beta} f(ga).$$

Each abstraction takes one input. $CIIa \rightarrow_{\beta}^* I(Ia) \rightarrow_{\beta}^* a$. Also $CII \rightarrow_{\beta}^* \lambda x. I(Ix) \rightarrow_{\beta}^* \lambda x. x = I$. Reductions under abstractions are allowed when proving beta-convertibility.

Executable illustration (Python):

```
I = lambda x: x
K = lambda x: lambda y: x
F = lambda x: lambda y: y
C = lambda f: lambda g: lambda x: f(g(x))
assert (I(7), K(7)(8), F(7)(8), C(I)(I)(7)) == (7, 7, 8, 7)
```

The same warm-ups in C++17:

```
#include <cassert>
int main() {
    auto I = [](int x) { return x; };
    auto K = [](int x) { return [x](int) { return x; }; };
    auto F = [](int) { return [](int y) { return y; }; };
    auto C = [](auto f) {
        return [f, g](int x) { return f(g(x)); }; };
};
assert(I(7) == 7 && K(7)(8) == 7 && F(7)(8) == 8);
assert(C(I)(I)(7) == 7);
}
```

These functions illustrate application with integer inputs; integers here belong to the host languages, not to the pure calculus.

2 Linked ideas: proofs and encodings

6. Boolean selectors

Key insight: A Boolean is a function that selects one of two branches. Let $T = \lambda a. \lambda b. a$ and $F = \lambda a. \lambda b. b$. Choose

$$\text{AND} = \lambda p. \lambda q. p q F, \quad \text{OR} = \lambda p. \lambda q. p T q, \quad \text{NOT} = \lambda p. p F T.$$

All are closed after expanding abbreviations. From $Tab \rightarrow_{\beta}^* a$ and $Fab \rightarrow_{\beta}^* b$ we get the complete truth tables:

p	q	$\text{AND } p q$	$\text{OR } p q$
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

For both rows with $p = T$, AND returns q and OR returns T ; for both rows with $p = F$, AND returns F and OR returns q . This proves every row independently of variable names. Also $\text{NOT } T \rightarrow_{\beta}^* F$ and $\text{NOT } F \rightarrow_{\beta}^* T$. Applying a selector to arbitrary a and b acts as a conditional without new syntax.

7. Church numerals and successor

Key insight: A numeral is an iterator, not an integer literal.

$$\bar{0} = \lambda f.\lambda x.x, \quad \bar{1} = \lambda f.\lambda x.fx, \quad \bar{2} = \lambda f.\lambda x.f(fx).$$

Set $\text{SUCC} = \lambda n.\lambda f.\lambda x.f(nfx)$. Expanding $\bar{n}$ gives

$$\text{SUCC } \bar{n} \rightarrow_{\beta}^* \lambda f.\lambda x.f(\bar{n}fx) \rightarrow_{\beta}^* \lambda f.\lambda x.f(f^n(x)) = \overline{n+1}.$$

By the recursive definition of f^n , this holds for every $n \geq 0$ (base $n = 0$: $f(x)$; inductive step adds one outer f). In particular $\bar{2}ga \rightarrow_{\beta} (\lambda x.g(gx))a \rightarrow_{\beta} g(ga)$.

8. Arithmetic as iteration

Key insight: Addition concatenates iterations; multiplication composes iterators. Take

$$\text{ADD} = \lambda m.\lambda n.\lambda f.\lambda x.mf(nfx), \quad \text{MUL} = \lambda m.\lambda n.\lambda f.m(nf), \quad \text{POW} = \lambda m.\lambda n.\lambda f.\lambda x.nmf x.$$

For arbitrary f, x , ADD yields $f^m(f^n(x)) = f^{m+n}(x)$, where the identity follows by induction on m . MUL yields $(f^n)^m(x) = f^{mn}(x)$, by induction on m using $n(m+1) = nm+n$. Finally $\bar{n} \bar{m} f x$ iterates the iterator $\bar{m}$ a total of n times, applying f m^n times. At $n = 0$ it yields fx , including $0^0 = 1$ under the stated convention. The outer $\lambda f.\lambda x$ in POW ensures that even for $n = 0$ the resulting term is beta-convertible to the *canonical* numeral $\bar{1}$: without this eta expansion $\bar{0} \bar{m} \rightarrow_{\beta} \lambda x.x$, which is not beta-convertible to $\bar{1}$.

Numeral iteration in Python:



```
def numeral(n):
    def apply(f):
        def run(x):
            for _ in range(n):
                x = f(x)
            return x
        return run
    return apply

def succ(n): return lambda f: lambda x: f(n(f)(x))
def add(m, n): return lambda f: lambda x: m(f)(n(f)(x))
def mul(m, n): return lambda f: m(n(f))
value = lambda church: church(lambda x: x + 1)(0)
assert [value(numeral(0)), value(succ(numeral(2))),
        value(add(numeral(2), numeral(3))),
        value(mul(numeral(2), numeral(3)))] == [0, 3, 5, 6]
```

The same encodings in C++17:

```
#include <cassert>
#include <functional>
using Fn = std::function<int(int)>;
using Church = std::function<Fn(Fn)>;
Church numeral(int n) {
    return [n](Fn f) -> Fn {
        return [n, f](int x) {
            for (int i = 0; i < n; ++i) x = f(x);
            return x;
        };
    };
}
Church succ(Church n) {
    return [n](Fn f) -> Fn {
        return [n, f](int x) { return f(n(f)(x)); };
    };
}
```

```

    };
}
Church add(Church m, Church n) {
    return [m, n](Fn f) -> Fn {
        return [m, n, f](int x) { return m(f)(n(f)(x)); };
    };
}
Church mul(Church m, Church n) {
    return [m, n](Fn f) -> Fn { return m(n(f)); };
}
int main() {
    auto value = [] (Church c) { return c([] (int x) { return x + 1; })(0); };
    assert(value(numeral(0)) == 0);
    assert(value(succ(numeral(2))) == 3);
    assert(value(add(numeral(2), numeral(3))) == 5);
    assert(value(mul(numeral(2), numeral(3))) == 6);
}

```

The iterated `int` functions are small executable observations; the preceding equational argument proves the identities for any f and x .

9. Subtraction by state threading

Key insight: To remember the previous numeral, iterate a pair of consecutive numerals. For a pair $p = \langle a, b \rangle$, the selector laws give $\text{FIRST } p \equiv_{\beta} a$ and $\text{SECOND } p \equiv_{\beta} b$. Thus $\text{STEP} \langle a, b \rangle \equiv_{\beta} \langle b, \text{SUCC } b \rangle$. Let $P_n = \text{STEP}^n \langle \bar{0}, \bar{0} \rangle$. $P_0 = \langle \bar{0}, \bar{0} \rangle$, which matches $\max(-1, 0) = 0$. For $n = 0$, $P_1 = \langle \bar{0}, \bar{1} \rangle$. If $n \geq 1$ and $P_n \equiv_{\beta} \langle \overline{n-1}, \bar{n} \rangle$, then $P_{n+1} \equiv_{\beta} \langle \bar{n}, \overline{n+1} \rangle$. This completes induction, including the exceptional first step. Since a Church numeral iterates a function,

$$\text{PRED} = \lambda n. \text{FIRST } (n \text{ STEP } (\text{PAIR } \bar{0} \bar{0}))$$

represents $\max(n-1, 0)$; in particular $\text{PRED } \bar{0} \equiv_{\beta} \bar{0}$. Set $\text{SUB} = \lambda m. \lambda n. n \text{ PRED } m$. Induction on n proves that n applications of truncated predecessor give $\max(m-n, 0)$, including $m < n$ and $m = 0$.

10. Logic meets numbers

Key insight: Iterators can repeatedly update truth values as well as numbers. Let $\text{ISZERO} = \lambda n. n (\lambda z. F) T$. For zero, there are no updates, so the answer is T . For $n+1$ there is at least one application of the constant- F function, and every subsequent application stays F ; induction on n proves the claim. Let $\text{EVEN} = \lambda n. n \text{ NOT } T$. Its output starts at T and alternates T/F after each step. Base $n = 0$ is T ; if the claim holds for n , NOT flips the answer for $n+1$, proving parity by induction. Finally $\text{SUB } \bar{m} \bar{n}$ is zero iff $m \leq n$, so ISZERO of that term selects T exactly in that case. This is a comparison on natural-number encodings, not an equality test on arbitrary lambda terms.

Optional slide challenge (division). Define $\text{GE} = \lambda a. \lambda b. \text{ISZERO}(\text{SUB } b a)$, selecting T iff $a \geq b$, and let $\text{IF} = \lambda c. \lambda a. \lambda b. cab$. For a fixed divisor d , define the closed-after-application step

$$\begin{aligned} \text{DSTEP} = & \lambda d. \lambda p. \text{IF } (\text{GE } (\text{SUCC}(\text{SECOND } p)) d) \\ & (\text{PAIR } (\text{SUCC}(\text{FIRST } p)) (\text{SUB } (\text{SUCC}(\text{SECOND } p)) d)) \\ & (\text{PAIR } (\text{FIRST } p) (\text{SUCC}(\text{SECOND } p))). \end{aligned}$$

Set

$$\text{DIV} = \lambda m. \lambda d. (\text{ISZERO } d) \bar{0} (\text{FIRST } (m (\text{DSTEP } d) (\text{PAIR } \bar{0} \bar{0}))).$$

For $d > 0$, induction on the number k of processed iterations shows that the state is $\langle \lfloor k/d \rfloor, \overline{k \bmod d} \rangle$. At $k = 0$ it is $\langle 0, 0 \rangle$. If the remainder $r < d-1$, the next step increments r ; if $r = d-1$, it increments the quotient and subtracts d from $r+1 = d$, resetting the remainder to zero. The selector returns the quotient after m iterations. For $d = 0$ DIV returns $\bar{0}$ by convention. The unused branch may be constructed but is not evaluated by normal-order beta reduction; no recursive fixed point is needed.

3 Extended homework: C++ implementations

11. Church arithmetic on a stack

Key insight: A residue map is a homomorphism for successor, addition and multiplication. Maintain the invariant that a stack entry equals the value of its Church numeral modulo $P = 10^9 + 7$. Z pushes zero, S adds one, A adds residues and M multiplies residues. The proofs for SUCC/ADD/MUL above establish that each local update preserves the invariant; induction over all tokens proves the output. Each operation takes $O(1)$ time, total $O(q)$ time and $O(q)$ space. A product of two residues fits in signed 64 bits.

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
int main() {
    constexpr long long P = 1000000007;
    int q; cin >> q;
    vector<long long> st;
    for (int i = 0; i < q; ++i) {
        string op; cin >> op;
        if (op == "Z") st.push_back(0);
        else if (op == "S") st.back() = (st.back() + 1) % P;
        else {
            long long a = st.back(); st.pop_back();
            long long b = st.back(); st.pop_back();
            st.push_back(op == "A" ? (b + a) % P : b * a % P);
        }
    }
    cout << st.back() << '\n';
}
```

12. Typed postfix booleans and numbers

Key insight: Modulo residues alone cannot tell whether a natural number is zero. For a number retain (r, z) with $r = n \bmod P$ and z true iff $n = 0$ (*exactly*). Z creates $(0, \text{true})$; S sets z false; A sets $z = z_b \wedge z_a$; M sets $z = z_b \vee z_a$. These tests follow from nonnegativity. Booleans retain the selector's T/F meaning. Z? reads the exact zero flag, NOT flips it, D/R implement the selector truth tables, and I selects the indicated number without changing it. Induction on the program prefix proves that each typed stack entry represents its corresponding lambda term and the output is correct even when n is a positive multiple of P . Time $O(q)$ and space $O(q)$.

```
#include <iostream>
#include <string>
#include <vector>
using namespace std;
struct Value {
    bool is_bool, bit, zero;
    long long residue;
};
int main() {
    constexpr long long P = 1000000007;
    int q; cin >> q;
    vector<Value> st;
    auto pop = [&]() { Value x = st.back(); st.pop_back(); return x; };
    auto num = [](long long r, bool z) { return Value{false, false, z, r}; };
    auto boolean = [](bool b) { return Value{true, b, false, 0}; };
    for (int i = 0; i < q; ++i) {
        string op; cin >> op;
        if (op == "Z") st.push_back(num(0, true));
        else if (op == "T" || op == "F") st.push_back(boolean(op == "T"));
    }
}
```

```
else if (op == "S") {
    Value a = pop(); st.push_back(num((a.residue + 1) % P, false));
} else if (op == "Z?") {
    Value a = pop(); st.push_back(boolean(a.zero));
} else if (op == "N") {
    Value a = pop(); st.push_back(boolean(!a.bit));
} else if (op == "I") {
    Value b = pop(), a = pop(), c = pop();
    st.push_back(c.bit ? a : b);
} else {
    Value a = pop(), b = pop();
    if (op == "A") st.push_back(num((b.residue + a.residue) % P,
                                    b.zero && a.zero));
    if (op == "M") st.push_back(num(b.residue * a.residue % P,
                                    b.zero || a.zero));
    if (op == "D") st.push_back(boolean(b.bit && a.bit));
    if (op == "R") st.push_back(boolean(b.bit || a.bit));
}
}
cout << st.back().residue << '\n';
}
```

